CAR 파일 페처 예제

개요

이 예제는 Trustless Gateway 사양을 구현하는 게이트웨이에서 UnixFS 파일이나 디렉토리를 신뢰할 수 있고 검증 가능한 방식으로 다운로드하는 방법을 보여줍니다. CAR(Content Addressable aRchive) 형식을 사용하여 단일 요청으로 전체 파일 구조를 가져오고 로컬에서 검증합니다.

이 예제를 통해 다음을 학습할 수 있습니다:

- CAR 파일 형식을 사용한 신뢰할 수 있는 데이터 가져오기
- IPNS 및 DNSLink 해석을 통한 가변 경로 처리
- 파일 크기 제한 및 사용자 에이전트 설정
- 검증 가능한 다운로드 구현

주요 개념

전체 흐름

- **CAR 페처 클라이언트**
  - 경로 해석 (IPNS / DNS)
  - CAR 요청 전송
- **Trustless Gateway**
  - CAR 생성 및 전송
  - CAR 응답 반환
- **로컬 검증 및 추출**

- 블록 검증
- 블록 재구성

핵심 구성 요소

1. **Remote CAR Backend**: 원격 게이트웨이와 통신하여 CAR 파일 요청
2. **Path Resolution**: IPNS 및 DNSLink 경로를 불변 경로로 해석
3. **Verification**: 다운로드된 블록의 암호학적 검증
4. **UnixFS Extraction**: CAR에서 UnixFS 파일/디렉토리 추출

완전한 구현 예제

1. 기본 설정 및 의존성

```go
package main


import (
"context"
"flag"
"fmt"
"io"
"log"
"net/http"
"os"
"path/filepath"
```

```
"time"


"github.com/ipfs/boxo/files"

"github.com/ipfs/boxo/gateway"

"github.com/ipfs/boxo/path"

"github.com/ipfs/go-cid"

)


const (

defaultGateway = "https://trustless-gateway.link"

defaultTimeout = 30 * time.Second

maxRetries = 3

)
```

2. CAR 페처 구조체 정의

```
// CARFetcher는 CAR 파일을 가져오고 검증하는 클라이언트입니다

type CARFetcher struct {

gatewayURL  string

httpClient *http.Client

backend gateway.IPFSBackend

userAgent  string

sizeLimit  int64
```

```
verbose bool
}

// FetchConfig는 페처 설정을 정의합니다
type FetchConfig struct {
GatewayURL string
UserAgent string
SizeLimit int64
Timeout time.Duration
Verbose bool
}

// NewCARFetcher는 새로운 CAR 페처를 생성합니다
func NewCARFetcher(config FetchConfig) (*CARFetcher, error) {
if config.GatewayURL == "" {
config.GatewayURL = defaultGateway
}
if config.Timeout == 0 {
config.Timeout = defaultTimeout
}

// 사용자 정의 HTTP 클라이언트 생성
httpClient := &http.Client{
```

```
Timeout: config.Timeout,

Transport: &limitedTransport{

RoundTripper: http.DefaultTransport,

limitBytes: config.SizeLimit,

userAgent: config.UserAgent,

},

}



// 원격 CAR 백엔드 생성

backend, err := gateway.NewRemoteCarBackend([]string{config.GatewayURL}, httpCli

if  err != nil {

return  nil, fmt.Errorf("CAR 백엔드 생성 실패: %w", err)

}



return &CARFetcher{

gatewayURL: config.GatewayURL,

httpClient: httpClient,

backend: backend,

userAgent: config.UserAgent,

sizeLimit: config.SizeLimit,

verbose: config.Verbose,

}, nil
```

```
}
```

3. 핵심 페치 기능 구현

```
// FetchResult는 페치 결과를 나타냅니다
type FetchResult struct {
Path path.Path
ResolvedPath path.Path
Size int64
BlockCount int
OutputPath string
}

// Fetch는 지정된 IPFS 경로에서 파일을 가져옵니다
func (cf *CARFetcher) Fetch(ctx context.Context, ipfsPath, outputPath string) (
if cf.verbose {
log.Printf("페치 시작: %s -> %s", ipfsPath, outputPath)
}

// IPFS 경로 파싱 및 검증
p, err := path.NewPath(ipfsPath)
if err != nil {
return nil, fmt.Errorf("잘못된 IPFS 경로: %w", err)
```

```
}

if cf.verbose {
log.Printf("경로 파싱 완료: %s", p.String())
}

// 가변 경로(IPNS, DNSLink) 해석
resolvedPath, _, _, err := cf.backend.ResolveMutable(ctx, p)
if err != nil {
return nil, fmt.Errorf("경로 해석 실패: %w", err)
}

if cf.verbose {
log.Printf("경로 해석 완료: %s -> %s", p.String(), resolvedPath.String())
}

// 파일 또는 디렉토리 가져오기
// 이 호출은 내부적으로 원격 게이트웨이에서 CAR 파일을 가져오고
// 모든 블록이 존재하고 검증되었는지 확인합니다
_, file, err := cf.backend.GetAll(ctx, resolvedPath)
if err != nil {
return nil, fmt.Errorf("파일 가져오기 실패: %w", err)
}
```

```
defer file.Close()

if cf.verbose {
log.Printf("CAR 파일 다운로드 및 검증 완료")
}

// 파일 시스템에 쓰기
if err := files.WriteTo(file, outputPath); err != nil {
return nil, fmt.Errorf("파일 쓰기 실패: %w", err)
}

// 결과 정보 수집
result := &FetchResult{
Path: p,
ResolvedPath: resolvedPath,
OutputPath: outputPath,
}

// 파일 크기 계산
if stat, err := os.Stat(outputPath); err == nil {
if stat.IsDir() {
result.Size = cf.calculateDirSize(outputPath)
} else {
```

```
result.Size = stat.Size()

}

}


if cf.verbose {

log.Printf("페치 완료: %d bytes", result.Size)

}


return result, nil

}


// calculateDirSize는 디렉토리의 총 크기를 계산합니다

func (cf *CARFetcher) calculateDirSize(dirPath string) int64 {

var size int64

filepath.Walk(dirPath, func(path string, info os.FileInfo, err error) error {

if err == nil && !info.IsDir() {

size += info.Size()

}

return nil

})

return size

}
```

4. 제한된 전송 구현

```
// limitedTransport는 크기 제한과 사용자 에이전트를 지원하는 HTTP 전송입니다
type limitedTransport struct {
http.RoundTripper
limitBytes  int64
userAgent  string
}

func (lt *limitedTransport) RoundTrip(req *http.Request) (*http.Response, error)
// 사용자 에이전트 설정
if  lt.userAgent != "" {
req.Header.Set("User-Agent", lt.userAgent)
}

// 요청 실행
resp, err := lt.RoundTripper.RoundTrip(req)
if  err != nil {
return  resp, err
}

// 응답 본문에 크기 제한 적용
if  resp != nil && resp.Body != nil && lt.limitBytes > 0 {
```

```
resp.Body = &limitReadCloser{
limit: lt.limitBytes,
ReadCloser: resp.Body,
}
}

return resp, err
}

// limitReadCloser는 읽기 크기를 제한하는 ReadCloser입니다
type limitReadCloser struct {
io.ReadCloser
limit int64
bytesRead int64
}

func (lrc *limitReadCloser) Read(p []byte) (int, error) {
n, err := lrc.ReadCloser.Read(p)
lrc.bytesRead += int64(n)
if lrc.bytesRead > lrc.limit {
return 0, fmt.Errorf("읽기 제한 초과: %d bytes 제한에서 %d bytes 읽음",
lrc.limit, lrc.bytesRead)
}
```

```
return n, err
}
```

5. 재시도 및 오류 복구

```
// FetchWithRetry는 재시도 로직을 포함한 페치를 수행합니다
func (cf *CARFetcher) FetchWithRetry(ctx context.Context, ipfsPath, outputPath
var lastErr error
for attempt := 0; attempt < maxRetries; attempt++ {
if attempt > 0 {
// 지수 백오프로 지연
delay := time.Duration(1<<uint(attempt-1)) * time.Second
if cf.verbose {
log.Printf("재시도 %d/%d, %v 후 재시도", attempt+1, maxRetries, delay)
}
select {
case <-time.After(delay):
case <-ctx.Done():
return nil, ctx.Err()
}
}
result, err := cf.Fetch(ctx, ipfsPath, outputPath)
if err == nil {
```

```
if attempt > 0 && cf.verbose {
log.Printf("재시도 성공 (시도 %d)", attempt+1)
}
return result, nil
}
lastErr = err
if cf.verbose {
log.Printf("시도 %d 실패: %v", attempt+1, err)
}
// 컨텍스트 취소 시 즉시 종료
if ctx.Err() != nil {
return nil, ctx.Err()
}
}
return nil, fmt.Errorf("모든 재시도 실패, 마지막 오류: %w", lastErr)
}
```

6. 배치 다운로드 지원

```
// BatchFetchConfig는 배치 다운로드 설정입니다
type BatchFetchConfig struct {
Paths []string
OutputDir string
```

```
Concurrency int
SkipErrors bool
}

// BatchFetchResult는 배치 다운로드 결과입니다
type BatchFetchResult struct {
Successful []FetchResult
Failed []BatchError
TotalSize int64
}

type BatchError struct {
Path string
Error error
}

// BatchFetch는 여러 파일을 동시에 다운로드합니다
func (cf *CARFetcher) BatchFetch(ctx context.Context, config BatchFetchConfig) (
if config.Concurrency <= 0 {
config.Concurrency = 3 // 기본값
}

// 출력 디렉토리 생성
```

```go
if err := os.MkdirAll(config.OutputDir, 0755); err != nil {
return nil, fmt.Errorf("출력 디렉토리 생성 실패: %w", err)
}

// 작업 채널 생성
jobs := make(chan string, len(config.Paths))
results := make(chan *FetchResult, len(config.Paths))
errors := make(chan BatchError, len(config.Paths))

// 워커 시작
for i := 0; i < config.Concurrency; i++ {
go cf.batchWorker(ctx, config.OutputDir, jobs, results, errors)
}

// 작업 전송
for _, path := range config.Paths {
jobs <- path
}
close(jobs)

// 결과 수집
result := &BatchFetchResult{}
for i := 0; i < len(config.Paths); i++ {
```

```
select {
case fetchResult := <-results:
if fetchResult != nil {
result.Successful = append(result.Successful, *fetchResult)
result.TotalSize += fetchResult.Size
}
case batchError := <-errors:
result.Failed = append(result.Failed, batchError)
if !config.SkipErrors {
return result, fmt.Errorf("배치 다운로드 실패: %w", batchError.Error)
}
case <-ctx.Done():
return result, ctx.Err()
}
}

return result, nil
}

// batchWorker는 배치 다운로드 워커입니다
func (cf *CARFetcher) batchWorker(ctx context.Context, outputDir string, jobs <
results chan<- *FetchResult, errors chan<- BatchError) {
for ipfsPath := range jobs {
```

```
// 출력 파일명 생성
outputPath := filepath.Join(outputDir, cf.generateFileName(ipfsPath))
result, err := cf.FetchWithRetry(ctx, ipfsPath, outputPath)
if err != nil {
errors <- BatchError{Path: ipfsPath, Error: err}
} else {
results <- result
}
}
}


// generateFileName은 IPFS 경로에서 파일명을 생성합니다
func (cf *CARFetcher) generateFileName(ipfsPath string) string {
// CID 추출 시도
p, err := path.NewPath(ipfsPath)
if err != nil {
return "unknown"
}
segments := p.Segments()
if len(segments) > 1 {
return segments[1] // CID 부분
}
return "fetched_file"
```

```
}
```

7. 메인 함수 및 CLI 인터페이스

```
func main() {
// 명령줄 플래그 정의
var (
gatewayURL = flag.String("g", defaultGateway, "CAR 파일을 다운로드할 trustless 게이
userAgent = flag.String("u", "", "HTTP 요청 시 사용할 사용자 에이전트")
outputPath = flag.String("o", "out", "가져온 경로를 저장할 출력 경로")
sizeLimit = flag.Int64("l", 0, "게이트웨이 다운로드 파일 크기 제한 (bytes)")
timeout = flag.Duration("t", defaultTimeout, "요청 타임아웃")
verbose = flag.Bool("v", false, "상세 로그 출력")
batch = flag.Bool("b", false, "배치 모드 (여러 경로를 파일에서 읽기)")
concurrent = flag.Int("c", 3, "배치 모드에서 동시 다운로드 수")
help = flag.Bool("h", false, "도움말 표시")
)

flag.Usage = func() {
fmt.Println("CAR 파일 페처 - 검증 가능한 IPFS 파일 다운로드")
fmt.Println()
fmt.Println("사용법:")
```

```
    fmt.Println(" car-fetcher [옵션] <IPFS_경로>")

    fmt.Println(" car-fetcher [옵션] -b <경로_목록_파일>")

    fmt.Println()

    fmt.Println("옵션:")

    flag.PrintDefaults()

    fmt.Println()

    fmt.Println("예제:")

    fmt.Println(" # 단일 파일 다운로드")

    fmt.Println(" car-fetcher /ipfs/bafkreifzjut3te2nhyekklss27nh3k72ysco7y32koao5ee

    fmt.Println()

    fmt.Println(" # 사용자 정의 게이트웨이 사용")

    fmt.Println(" car-fetcher -g https://dweb.link /ipfs/QmHash...")

    fmt.Println()

    fmt.Println(" # 크기 제한과 함께")

    fmt.Println(" car-fetcher -l 10485760 /ipfs/QmHash...") // 10MB 제한

    fmt.Println()

    fmt.Println(" # 배치 다운로드")

    fmt.Println(" car-fetcher -b -c 5 paths.txt")

    }



    flag.Parse()



    if *help {
```

```
flag.Usage()
return
}

ctx, cancel := context.WithCancel(context.Background())
defer  cancel()

// CAR 페처 생성
config := FetchConfig{
GatewayURL: *gatewayURL,
UserAgent: *userAgent,
SizeLimit: *sizeLimit,
Timeout: *timeout,
Verbose: *verbose,
}

fetcher, err := NewCARFetcher(config)
if  err != nil {
log.Fatalf("페처 생성 실패: %v", err)
}

if *batch {
// 배치 모드
```

```
pathsFile := flag.Arg(0)

if pathsFile == "" {

log.Fatal("배치 모드에서는 경로 목록 파일이 필요합니다")

}



paths, err := readPathsFromFile(pathsFile)

if err != nil {

log.Fatalf("경로 파일 읽기 실패: %v", err)

}



batchConfig := BatchFetchConfig{

Paths: paths,

OutputDir: *outputPath,

Concurrency: *concurrent,

SkipErrors: true,

}



result, err := fetcher.BatchFetch(ctx, batchConfig)

if err != nil {

log.Fatalf("배치 다운로드 실패: %v", err)

}



// 결과 출력
```

```
fmt.Printf("배치 다운로드 완료:\n")

fmt.Printf(" 성공: %d개 파일, 총 %d bytes\n", len(result.Successful), result.Total

fmt.Printf(" 실패: %d개 파일\n", len(result.Failed))



for _, failed := range result.Failed {

fmt.Printf(" 실패: %s - %v\n", failed.Path, failed.Error)

}



} else {

// 단일 파일 모드

ipfsPath := flag.Arg(0)

if ipfsPath == "" {

flag.Usage()

os.Exit(1)

}



result, err := fetcher.FetchWithRetry(ctx, ipfsPath, *outputPath)

if err != nil {

log.Fatalf("페치 실패: %v", err)

}



fmt.Printf("다운로드 완료:\n")

fmt.Printf(" 경로: %s\n", result.Path.String())
```

```
fmt.Printf(" 해석된 경로: %s\n", result.ResolvedPath.String())

fmt.Printf(" 출력: %s\n", result.OutputPath)

fmt.Printf(" 크기: %d bytes\n", result.Size)

}

}



// readPathsFromFile은 파일에서 IPFS 경로 목록을 읽습니다

func readPathsFromFile(filename string) ([]string, error) {

file, err := os.Open(filename)

if err != nil {

return nil, err

}

defer file.Close()



var paths []string

scanner := bufio.NewScanner(file)

for scanner.Scan() {

line := strings.TrimSpace(scanner.Text())

if line != "" && !strings.HasPrefix(line, "#") {

paths = append(paths, line)

}

}
```

```
return paths, scanner.Err()

}
```

실행 방법

1. 빌드

```
# 프로젝트 디렉토리에서

go mod init car-fetcher-example

go mod tidy

go build -o car-fetcher
```

2. 단일 파일 다운로드

```
# 기본 게이트웨이에서 파일 다운로드

./car-fetcher /ipfs/bafkreifzjut3te2nhyekklss27nh3k72ysco7y32koao5eei66wof36n5e


# 사용자 정의 게이트웨이 사용

./car-fetcher -g https://dweb.link /ipfs/QmHash...


# 출력 경로 지정

./car-fetcher -o hello.txt /ipfs/bafkreifzjut3te2nhyekklss27nh3k72ysco7y32koa
```

```
# 크기 제한 설정 (10MB)

./car-fetcher -l 10485760 /ipfs/QmHash...
```

3. 디렉토리 다운로드

```
# 전체 디렉토리 다운로드

./car-fetcher -o my-website /ipfs/QmDirectoryHash...


# 상세 로그와 함께

./car-fetcher -v -o my-website /ipfs/QmDirectoryHash...
```

4. IPNS 및 DNSLink 지원

```
# IPNS 경로 다운로드

./car-fetcher /ipns/k51qzi5uqu5dgutdk6zqyp1qtqf1yt1rkgzhvtjkj1yt1rkgzhvtjkj1yt1


# DNSLink 경로 다운로드

./car-fetcher /ipns/ipfs.io
```

5. 배치 다운로드

```
# 경로 목록 파일 생성

cat > paths.txt << EOF
```

```
/ipfs/bafkreifzjut3te2nhyekklss27nh3k72ysco7y32koao5eei66wof36n5e
/ipfs/QmAnotherHash...
/ipns/example.com
EOF

# 배치 다운로드 실행
./car-fetcher -b -c 5 -o downloads paths.txt
```

고급 사용 사례

1. 웹사이트 미러링

```
# 전체 웹사이트 다운로드
./car-fetcher -v -o website-mirror /ipfs/bafybeiaysi4s6lnjev27ln5icwm6tueaw2

# 결과: 완전한 정적 웹사이트가 로컬에 저장됨
```

2. 대용량 데이터셋 다운로드

```
# 크기 제한과 재시도를 통한 안전한 다운로드
./car-fetcher -l 1073741824 -v /ipfs/QmLargeDataset... # 1GB 제한
```

3. 자동화된 백업

```
#!/bin/bash

# backup.sh - IPFS 콘텐츠 자동 백업 스크립트


BACKUP_DIR="/backup/ipfs-content"

PATHS_FILE="important-paths.txt"


mkdir -p "$BACKUP_DIR"


./car-fetcher -b -c 10 -o "$BACKUP_DIR" "$PATHS_FILE"


if [ $? -eq 0 ]; then

echo "백업 완료: $(date)"

# 백업 완료 알림 또는 추가 처리

else

echo "백업 실패: $(date)"

# 오류 알림

fi
```

성능 최적화

1. 동시 다운로드 조정

```
# CPU 코어 수에 따른 동시성 조정
```

```
CORES=$(nproc)

./car-fetcher -b -c $((CORES * 2)) paths.txt
```

2. 네트워크 대역폭 고려

```
# 대역폭이 제한된 환경에서는 동시성 감소

./car-fetcher -b -c 2 -l 52428800 paths.txt # 50MB 제한, 2개 동시
```

3. 메모리 사용량 최적화

```
# 큰 파일의 경우 크기 제한 설정

./car-fetcher -l 104857600 /ipfs/QmLargeFile... # 100MB 제한
```

장점과 특징

CAR 기반 페치의 장점

1. **효율성**: 단일 HTTP 요청으로 전체 파일 구조 다운로드
2. **검증 가능성**: 모든 블록의 암호학적 무결성 검증
3. **신뢰성**: 신뢰할 수 없는 게이트웨이에서도 안전한 다운로드
4. **완전성**: 부분적 다운로드 없이 완전한 파일 구조 보장

지원 기능

1. **경로 해석**: IPNS 및 DNSLink 자동 해석

2. **재시도 로직**: 네트워크 오류 시 자동 재시도
3. **크기 제한**: 예상치 못한 대용량 다운로드 방지
4. **배치 처리**: 여러 파일 동시 다운로드
5. **진행률 추적**: 상세한 로그 및 진행률 정보

모범 사례

1. 오류 처리

```
// 적절한 오류 메시지 제공
if err != nil {
log.Printf("다운로드 실패 (%s): %v", ipfsPath, err)
return fmt.Errorf("구체적인 오류 설명: %w", err)
}
```

2. 리소스 관리

```
// 파일 핸들 적절히 정리
defer file.Close()

// 컨텍스트 취소 처리
select {
case <-ctx.Done():
return ctx.Err()
```

```
default:

// 계속 진행

}
```

3. 로깅

```
// 구조화된 로깅 사용

if verbose {

log.Printf("진행 상황: %s -> %s (%d bytes)",

source, destination, size)

}
```

4. 설정 검증

```
// 입력 매개변수 검증

if config.SizeLimit < 0 {

return nil, fmt.Errorf("크기 제한은 0 이상이어야 합니다")

}
```

문제 해결

일반적인 문제와 해결책

1. **게이트웨이 연결 실패**

- 게이트웨이 URL 확인
- 네트워크 연결 상태 점검
- 다른 게이트웨이 시도

2. **CAR 형식 지원 오류**

- IPIP-0402 지원 게이트웨이 사용 확인
- 게이트웨이 호환성 확인

3. **크기 제한 초과**

- 적절한 크기 제한 설정
- 파일 크기 사전 확인
- 부분 다운로드 고려

4. **IPNS 해석 실패**

- IPNS 레코드 유효성 확인
- 게이트웨이의 IPNS 지원 확인
- DNSLink 설정 점검

5. **메모리 부족**

- 크기 제한 설정
- 배치 크기 조정
- 스트리밍 처리 고려

이 예제를 통해 신뢰할 수 있고 효율적인 IPFS 콘텐츠 다운로드 시스템을 구현할 수 있으며, 실제 프로덕션 환경에서는 추가적인 모니터링, 로깅, 보안 기능을 고려해야 합니다.